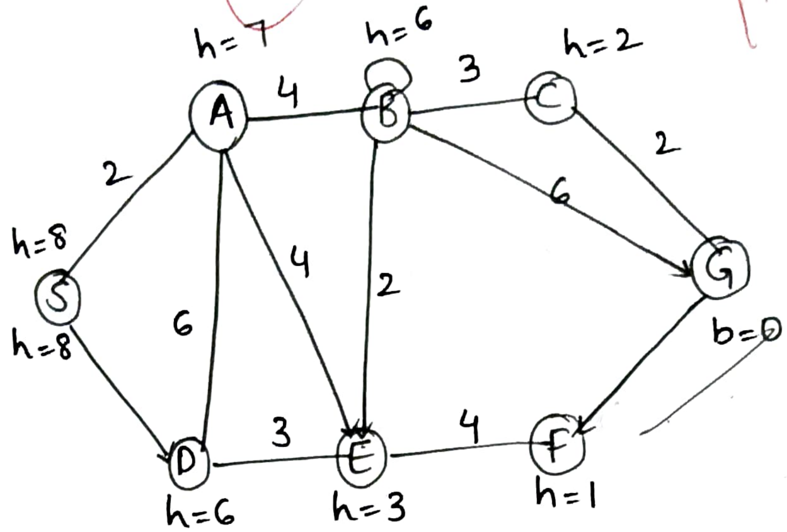
Namburi Sai Yashaswini 192425250

1.



Smart Ambulance Navigation (A* Algorithm)

Approach: use A* since both

Actual cost: g(n) (road cost)

Heuristic h(n) (estimated distance)

This is an Informed search problem

$$f(n) = g(n) + h(n)$$

Step by step procedure:

1. Initialize open list with start node
2. Calculate f(n) for all nodes
3. Select node with minimum f(n)
4. Expand node → Generate Successors

5. update costs and parent nodes
6. Repeat it until the Goal is reached

Example Execution (General form)

Node	g(n)	h(n)	f(n)
start	0	10	10
A	2	6	8
B	4	5	9

Choose A (f=8)

Start → A → Next best node → Goal

Optimality Justification.

→ A* is optimal if heuristic is
- ✓ Admissible
- ✓ Consistent

→ Hence Ambulance gets shortest & fastest route

Time Complexity: O(b^d)

Autonomous warehouse Robot (online search -L·RTA*)

Grid (S=Start, G=Goal, # =obstacle)

S	.	.	#	.
.	#	.	.	.
.	.	#	.	.
#	.	.	#	.
.	.	.	.	G

6

Initial heuristic:

7	6	5	4	3
6	5	4	3	2
5	4	3	2	1
4	3	2	1	1
3	2	1	1	0

The robot uses LRTA*

(Learning Real time A*)

At each step, it moves to neighbour with minimum h

If it hits a new obstacle it updates h(current) and chooses another path

Step	Current Node	Action	New obstacle	hvalue	Next move
1	S(1,1)	Right	NO	–	(1,2)
2	S(1,2)	Right	NO	–	(1,3)
3	S(1,3)	Right	Yes(+1)	h(1,3) =1+min (4,3)=4	Down
4	(2,3)	Down	NO	–	(3,3)
5	(3,3)	Right	NO	–	(3,4)
6	(3,4)	Down	NO	–	(4,4)
7	(4,4)	Down	NO	–	G(5,4)

Path taken by Robot:

S → (1,2) → (1,3) → (2,3) → (3,3) → (3,4) → (4,4) → G

Comparison:

Online Search (LRTA*)	Offline Search (A*)
→ Discovers obstacles during execution	→ Needs complete knowledge
→ updates heuristic values	→ Computes optimal Path before moving
→ Requires less memory	→ more time & memory
→ Suitable	→ Not suitable where map is unknown

University Examination scheduling CSP:

a) CSP formulation:

Variables : $E_1, E_2, E_3, E_4, E_5, E_6$

Domains : $D(E_i) = \{T_1, T_2, T_3, T_4\}$ for $i = 1$ to 6

Constraints (examinations with common students cannot be in same slot)

1. $E_1 \neq E_2$ 2. $E_1 \neq E_3$ 3. $E_2 \neq E_4$ 4. $E_3 \neq E_5$

5. $E_4 \neq E_6$ 6. $E_2 \neq 5$

Hall capacities

Hall A	120
Hall B	90
Hall C	60

No. of Students

Exam	Students	Exam	Students
E1	100	E4	60
E2	80	E5	90
E3	70	E6	50

b) Backtracking with MRV + Forward Checking

Initial Domains

Step 1: $E1 = T1$

Step 2: $E2 = T2$

Step 3 : $E3 = T2$

Step 4 : $E4 = T3$

Step 5: $E5 = T1$

Step 6: $E6 = T1$

E1	E2	E3	E4	E5	E6
T_1, T_2, T_4	T_1, T_2, T_4	T_1, T_3, T_4	T_1, T_3, T_4	T_1, T_3, T_4	$T_1, T_3,$
T_1	E2 T_2, T_5, T_4	E2 T_2, T_5, T_4	E4 T_2, T_3, T_4	E4 $T_2 T_3 T_4$	
T_1	T_2	–	E6 $T_1 T_3 T_4$		
T_1	T_2	T_3	T_4		
T_1	T_1	T_3			
T_1	T_1	T_1			

ime playin

c) Final timetable:

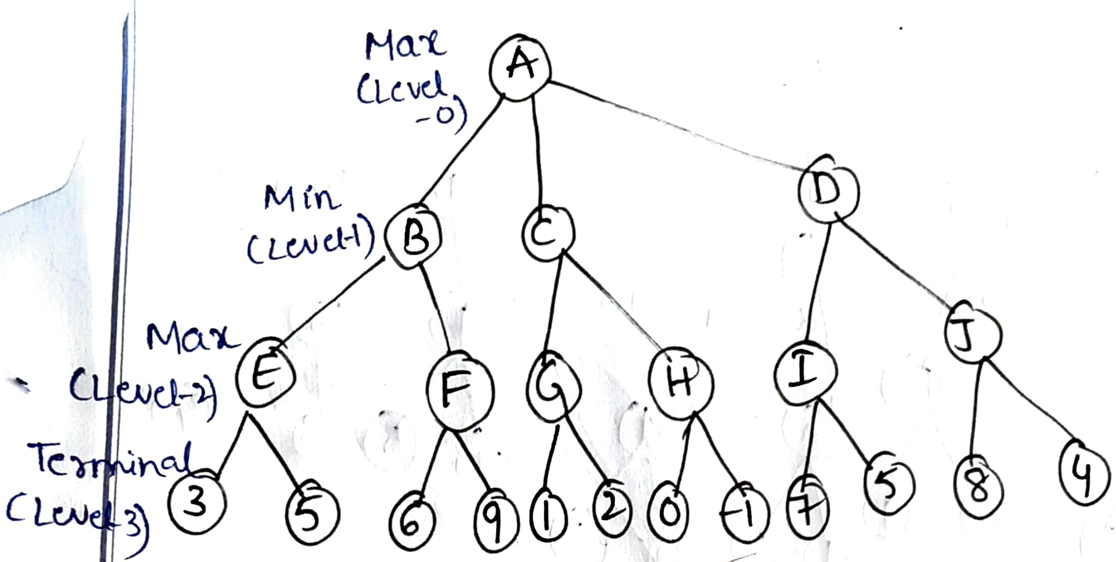
Time slot	Examinations
T1	E1, E5, E6
T2	E2, E3
T3	E4
T4	

Hall Allocation:

All constraints are Satisfied

Time	Hall A	Hall B	Hall C
T1	E1(100)	E5(90)	E6(50)
T2	E3(70)	E2(80)	–
T3	E4(60)	–	–
T4	–	–	–

Game playing AI (Minimax - αβ pruning)



Minimax (Bottomup)

$E = max(3,5) = 5$ $F = max(6,9) = 9$

$B = min(5,9) = 5$

$G = max(1,2) = 2$ $H = max(0,-1) = 0$

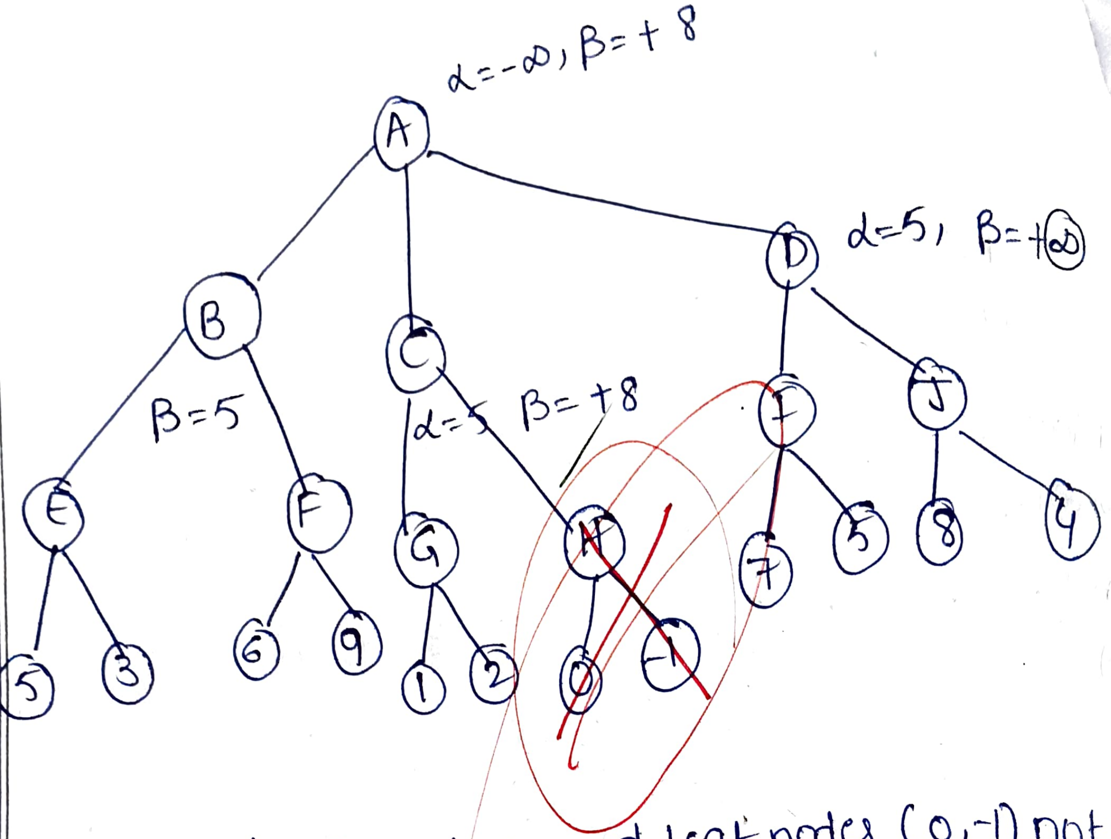
$C = min(2,0) = 0$

$I = max(7,5) = 7$ $J = max(8,4) = 8$

$D = Min(7,8) = 7$

$A = Max(B,C,D) = 7$

Alpha beta pruning:



Pruned: Branch H and leaf nodes (0,-1) not evaluated

Benefit: αβ pruning reduces no. of nodes evaluated from 10 to 8, thus Improving efficiency.

Delivery Route optimization (Hill Climbing)

Customers: A (Depot), B, C, D, E, F

	A	B	C	D	E	F
A	0	2	9	10	7	3
B	2	0	6	4	3	8
C	9	6	0	8	5	7
D	10	4	8	0	6	2
E	7	3	5	6	0	4
F	3	8	7	2	4	0

Initial Route: A – B – C – D – E – F – A

Total Distance: 2+6+8+6+4+3=29

Hill Climbing (2-opt Neighborhood)

Iteration	Current Route	Total	Best Neighbor	Distance
0	A-B-C-D-E-F-A	29	A-B-D-C-E-F-A	21
1	A-B-D-C-E-F-A	21	A-B-D-F-E-C-A	19
2	A-B-D-F-E-C-A	19	A-F-E-D-B-C-A	17
3	A-F-E-D-B-C-A	17	No better	-

Final Route:

A-F-E-D-B-C-A

Total Distance = 17

Is it Global Optimum:

There are $\frac{5!}{2} = 60$ possible tours

(since start direction is fixed).

Exhaustive check (or by inspection) shows no route with <17 Hence it is Global Optimum

How to escape Optimum:

- use Random-Restart Hill climbing
- use Simulated Annealing
- use Genetic Algorithm
- use Tabu search

These methods allow exploration of non improving moves and help escape local optima